类的继承 2

教　师：冀荣华

单　位：人工智能系

主要内容

一、继承与派生

二、派生类的声明方式

三、派生类的构成

四、派生类成员的访问属性

五、派生类的构造函数和析构函数

六、多继承

七、基类与派生类的转换

五、派生类的构造函数和析构函数

```
#include<iostream.h>
class Human
{ public:
      Human()
      {cout<<"a person is constructed!!!!\n";}
      ~Human()
      {cout<<"a person is deconstructed!!!!\n";}
};
class Student :public Human
{public:
      Student()
      {cout<<"a student is constructed!!!!\n";}
      ~Student()
      {cout<<"a student is deconstructed!!!!\n";}
};
void main()
{     Student jessic;
      cout<<"this is main function\n";
}
```

分析程序运行结果

总结派生类对象的生成过程

五、派生类的构造函数和析构函数

```
#include <iostream.h>
class Human
{ public:   Human( )
            {cout<<"a person is constructed!!!!\n";}
            ~Human()
            {cout<<"a person is deconstructed!!!!\n";}
};
class Student :public Human
{private: int id;
public:     Student(int i)
            {           id=i;
                        cout<<id<<"  student is constructed!!!!\n";}
            ~Student()
            {cout<<id<<" student is deconstructed!!!!\n";}
};
void main()
{    Student jessic(1234);
     cout<<"this is main function\n";  }
```

分析程序运行结果

继续总结派生类对象的生成过程

五、派生类的构造函数和析构函数

```
#include <iostream.h>
class Human
{ int age;
public:
    Human(int a )
    {       age=a;
            cout<<age<<"a person is constructed!!!!\n";}
    ~Human()
    {cout<<age<<“a person is deconstructed!!!!\n”;}      };
class Student :public Human
{private: int id;
public:     Student(int i)
    {       id=i;
            cout<<id<<"  student is constructed!!!!\n";}
    ~Student()
    {cout<<id<<" student is deconstructed!!!!\n";}
};
void main()
{   Student jessic(1234);
    cout<<"this is main function\n";   }
```

编译程序,看是否有错误出现?

思考其原因

```
#include <iostream.h>
class Human
{ int age;
public:    Human(int a )
    {      age=a;
           cout<<age<<"a person is constructed!!!!\n";}
    ~Human()
    {cout<<age<<"a person is deconstructed!!!!\n";}
};
class Student :public Human
{private:int id;
public:    Student(int i,int a):Human(a)
    {      id=i;
           cout<<id<<"  student is constructed!!!!\n";}
    ~Student()
    {cout<<id<<" student is deconstructed!!!!\n";}
};
void main()
{   Student jessic(1234,12);
    cout<<"this is main function\n";}
```

观察解决方案

请注意与组合类中初始化成员对象相区别

五、派生类的构造函数和析构函数

- 基类的构造函数不被继承，派生类中需要声明自己的构造函数。

- 声明构造函数时，只需要对本类中新增成员进行初始化，对继承来的基类成员的初始化，自动调用基类构造函数完成。

- 派生类的构造函数需要给基类的构造函数传递参数

五、派生类的构造函数和析构函数

派生类名::派生类名(基类所需形参，本类所需形参):基类名(基类参数表)

{

本类成员初始化赋值语句;

};

```
Student(int i,int a):Human(a)
```

```
#include "iostream.h"
class Human
{   int age;
public:     Human(int a )
            {age=a;                         cout<<age<<"a person is constructed!!!!\n";}
            ~Human()            {cout<<age<<"a person is deconstructed!!!!\n";}
 };
class Score
{ public:  Score()    {          cout<<"a score is constructed!!!!\n";              }
     ~Score()         {          cout<<“a score is deconstructed!!!!\n”;            }
};
class Student :public Human
{private:  int id;                          Score s;
public:     Student(int i,int a):Human(a)
            {          id=i;                cout<<id<<"  student is constructed!!!!\n";}
            ~Student()
            {cout<<id<<" student is deconstructed!!!!\n";}     };
void main()
{    Student jessic(1234,12);
     cout<<"this is main function\n";    }
```

分析程序运行结果

继续总结派生类对象的生成过程

```
#include "iostream.h"
class Human
{   int age;
public:     Human(int a )
            {age=a;                         cout<<age<<"a person is constructed!!!!\n";}
            ~Human()            {cout<<age<<"a person is deconstructed!!!!\n";}
 };
class Score
{ public:  Score()    {          cout<<"a score is constructed!!!!\n";              }
     ~Score()         {          cout<<“a score is deconstructed!!!!\n”;            }
};
class Student :public Human
{private:  int id;                          Score s;
public:     Student(int i,int a):Human(a)
            {          id=i;                cout<<id<<"  student is constructed!!!!\n";}
            ~Student()
            {cout<<id<<" student is deconstructed!!!!\n";}     };
void main()
{    Student jessic(1234,12);
     cout<<"this is main function\n";    }
```

分析程序运行结果

继续总结派生类对象的生成过程

```
#include "iostream.h"
class Human
{ int age;
public:     Human(int a )
            {age=a;                          cout<<age<<"a person is constructed!!!!\n";}
            ~Human()
            {cout<<age<<"a person is deconstructed!!!!\n";}          };
class Score
{ int s;
public:     Score(int i)
     {      s=i;   cout<<i<<" score is constructed!!!!\n";                    }
     ~Score()
     {      cout<<i<<" score is deconstructed!!!!\n";    }     };
class Student :public Human
{private:  int id;                          Score s;
public:     Student(int i ,int a):Human(a)
            {           id=i;                cout<<id<<"  student is constructed!!!!\n";}
            ~Student()
            {cout<<id<<" student is deconstructed!!!!\n";}     };
void main()
{    Student jessic(1234,12);
     cout<<"this is main function\n";    }
```

编译程序,看是否有错误出现?

思考其原因

```
#include "iostream.h"
class Human
{ int age;
public:      Human(int a )
             {age=a;                              cout<<age<<"a person is constructed!!!!\n";}
             ~Human()
             {cout<<age<<"a person is deconstructed!!!!\n";}          };
class Score
{ int s;
public:      Score(int i)
     {       s=i;   cout<<i<<" score is constructed!!!!\n";                              }
     ~Score()
     {       cout<<s<<"score is deconstructed!!!!\n";    }     };
class Student :public Human
{private:  int id;                                Score s;
public:      Student(int i,int a,int b):Human(a),s(b)
             {              id=i;                  cout<<id<<"  student is constructed!!!!\n";}
             ~Student()
             {cout<<id<<"student is deconstructed!!!!\n";}     };
void main()
{    Student jessic(1234,12,100);
     cout<<"this is main function\n";    }
```

观察解决方案

请注意派生组合类中初始化

构造函数的执行顺序

1. 基类构造函数
2. 成员对象的构造函数
3. 派生类的构造函数

五、派生类的构造函数和析构函数

析构函数的调用次序与构造函数严格相反

五、派生

```
#include <iostream>
#include<string>
using namespace std;
class Student
{ public:   Student(int n, string nam)
            {          num=n;              name=nam;   }
            void display()
            {          cout<<"num:"<<num<<endl;
                       cout<<"name:"<<name<<endl;   }
 protected:   int num;    string name;              };
class Student1:public Student
{ public:   Student1(int n,string nam,int a):Student(n,nam)
            {age=a; }
            void show()
            {          display();                  cout<<"age: "<<age<<endl;   }
 private:  int age;     };
class Student2:public Student1
{ public:   Student2(int n, string nam,int a,int s):Student1(n,nam,a)
            {          score=s;}
            void show_all()
            {          show();     cout<<"score:"<<score<<endl;      }
 private:   int score;  };
 void main()
 {          Student2 stud(10010,"Li",17,89);
            stud.show_all();      }
```

五、派生类的构造函数和析构函数

1　继承方式

必须掌握

采用不同的继承方式而派生出来的子类中成员的访问控制权限

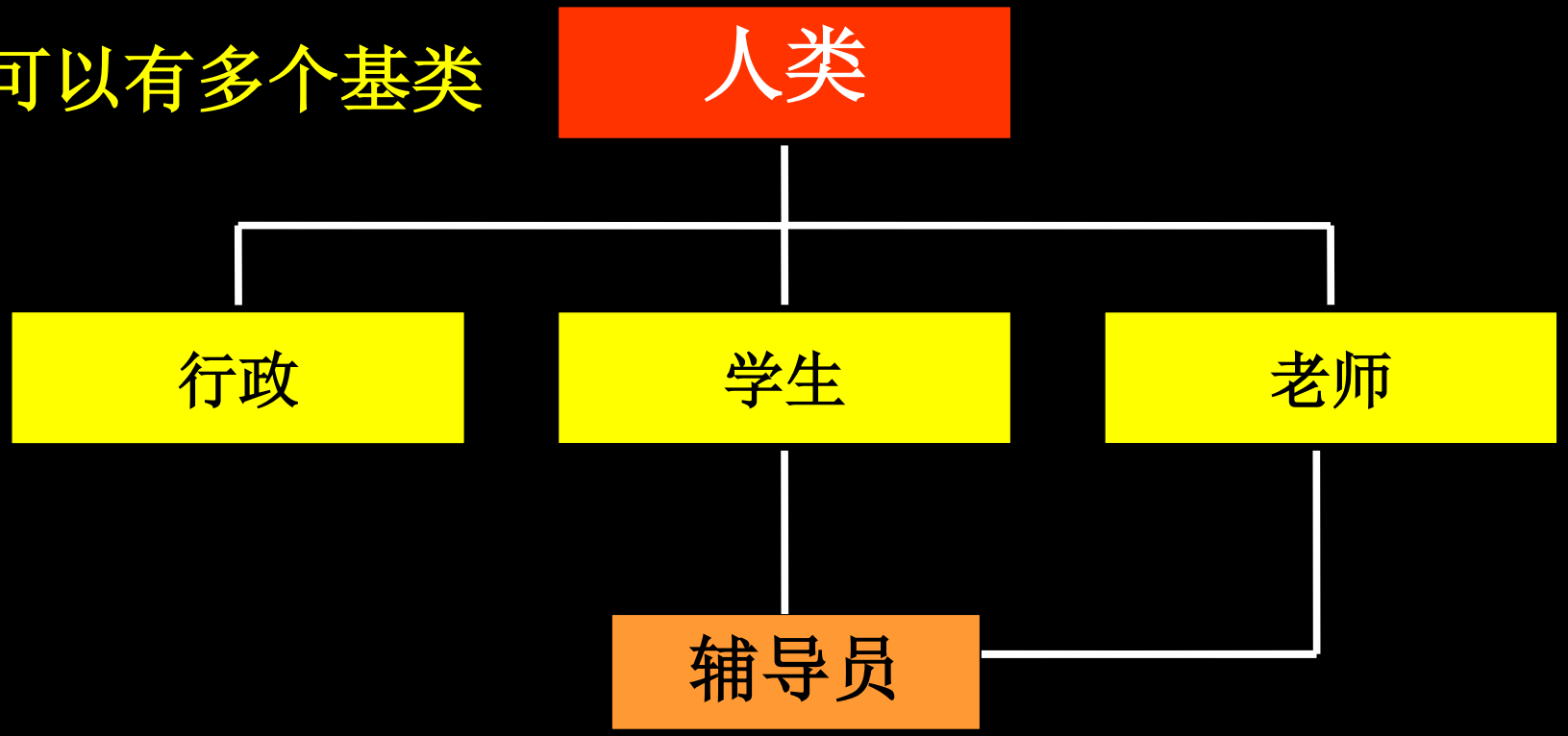
2 派生类的构造函数

　　如果基类中定义了带参数的构造函数

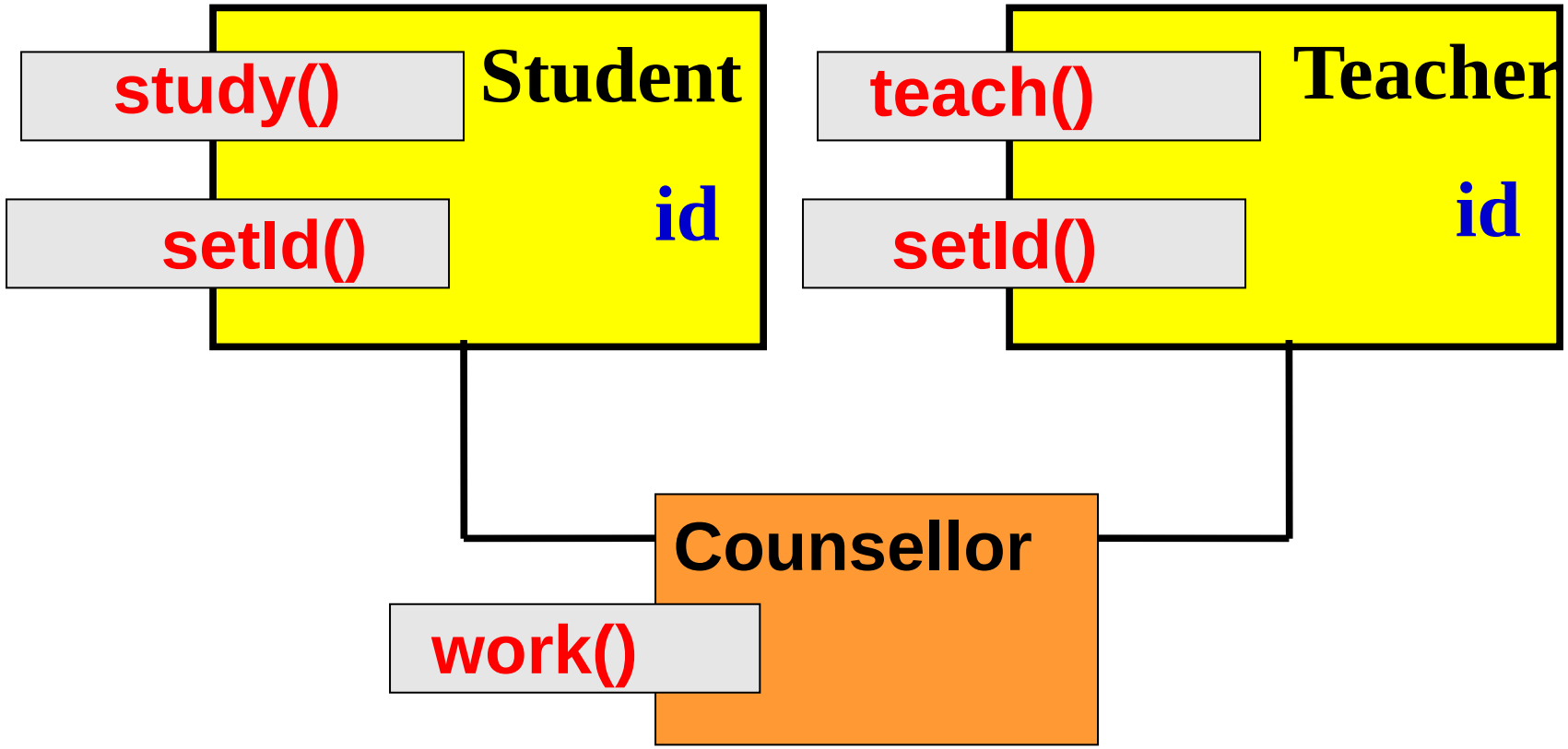
要求派生类须定义带参数构造函数，并为基类构造函数传递参数

六、多继承

一个派生类可以有多个基类



六、多继承


Student
study()
setId()
id
Teacher
teach()
setId()
id
Counsellor
work()

六、多继承

```
#include<iostream.h>
class Student
{   public:     Student() :id(0){}
                void study(){ cout <<"Studying...\n"; }
                void setId (int i){ id =i; }
     protected:   int id;

};
class Teacher
{  public:   Teacher()  {id=0;}
             void teach(){ cout <<"Teaching ....\n"; }
              void setId(int i){ id =i; }
    protected:    int id;
};
```

六、多继承

```
class Counsellor:public Student , public Teacher
{
    public:
        Counsellor (){}
        void work (){ cout <<"Woking ….\n"; }
};

void main()
{
    Counsellor ss;
    ss.study();
    ss.teach();
    ss.work();
}
```

六、多继承

```
#include<iostream.h>
class Student
{protected:          int id;
public:    Student()
           {id=0;}
           void study()
           { cout <<"Studying ...\n"; }
           void setId(int i)
           { id =i; }
};
class Teacher
{public:   Teacher():id(0)
           {}
           void teach(){ cout <<"Teaching...\n"; }
           void setId(int i){ id =i; }
protected:          int id;
};
class Counsellor:public Student, public Teacher
{public:   Counsellor(){}
           void work(){ cout <<"working...\n"; }
};
void main()
{          Counsellor ss;
           ss.study();  ss.teach();  ss.work();    }
```

六、多继承

说明1： 多继承时派生类的声明

```
class 派生类名:继承方式1 基类1,继承方式2 基类2,...
{
    成员声明;
}
```

注意：

每一个“继承方式”，只用于限制对紧随其后之基类的继承

说明2： 继承时的模糊性

Counsellor 类有两个id数据成员，那么：

ss.setId(20)；//设置的是哪个id的值

——名称冲突（name collision）,编译会出错

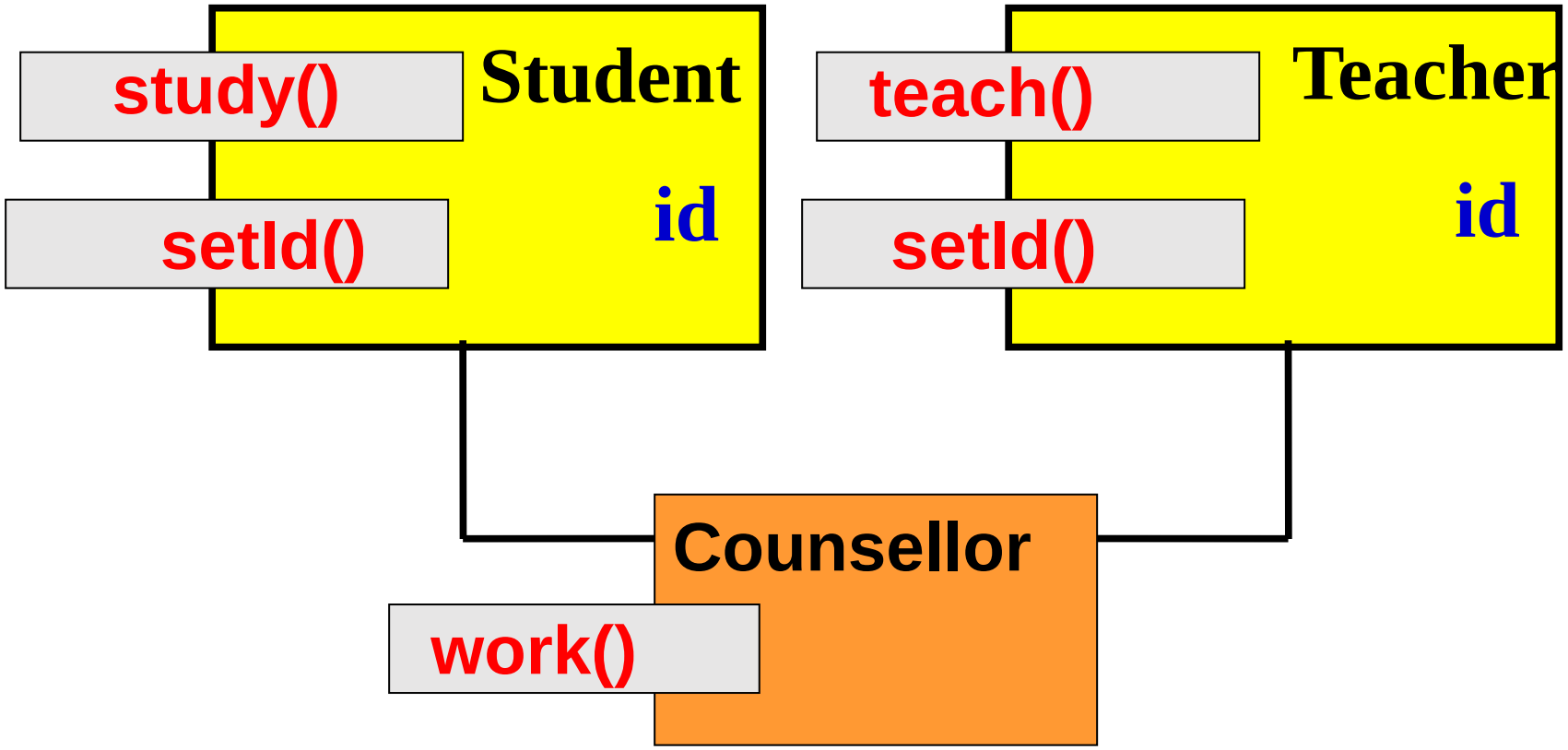
解决方案：

加上基类名加以说明来源

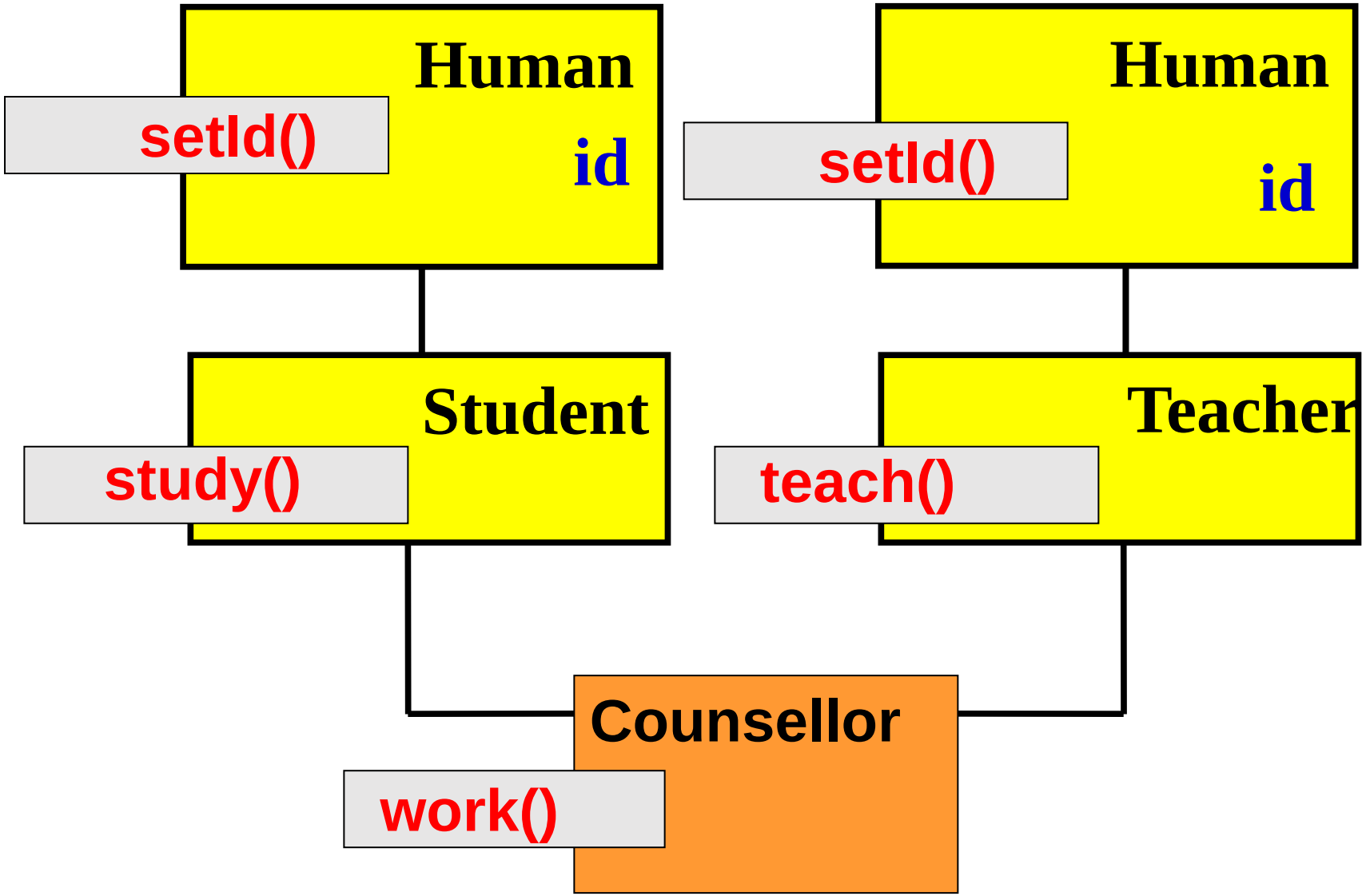
ss.Teacher::setId (20)；//设置Teacher的id值

六、多继承

说明3： 分析类的关系



六、多继承

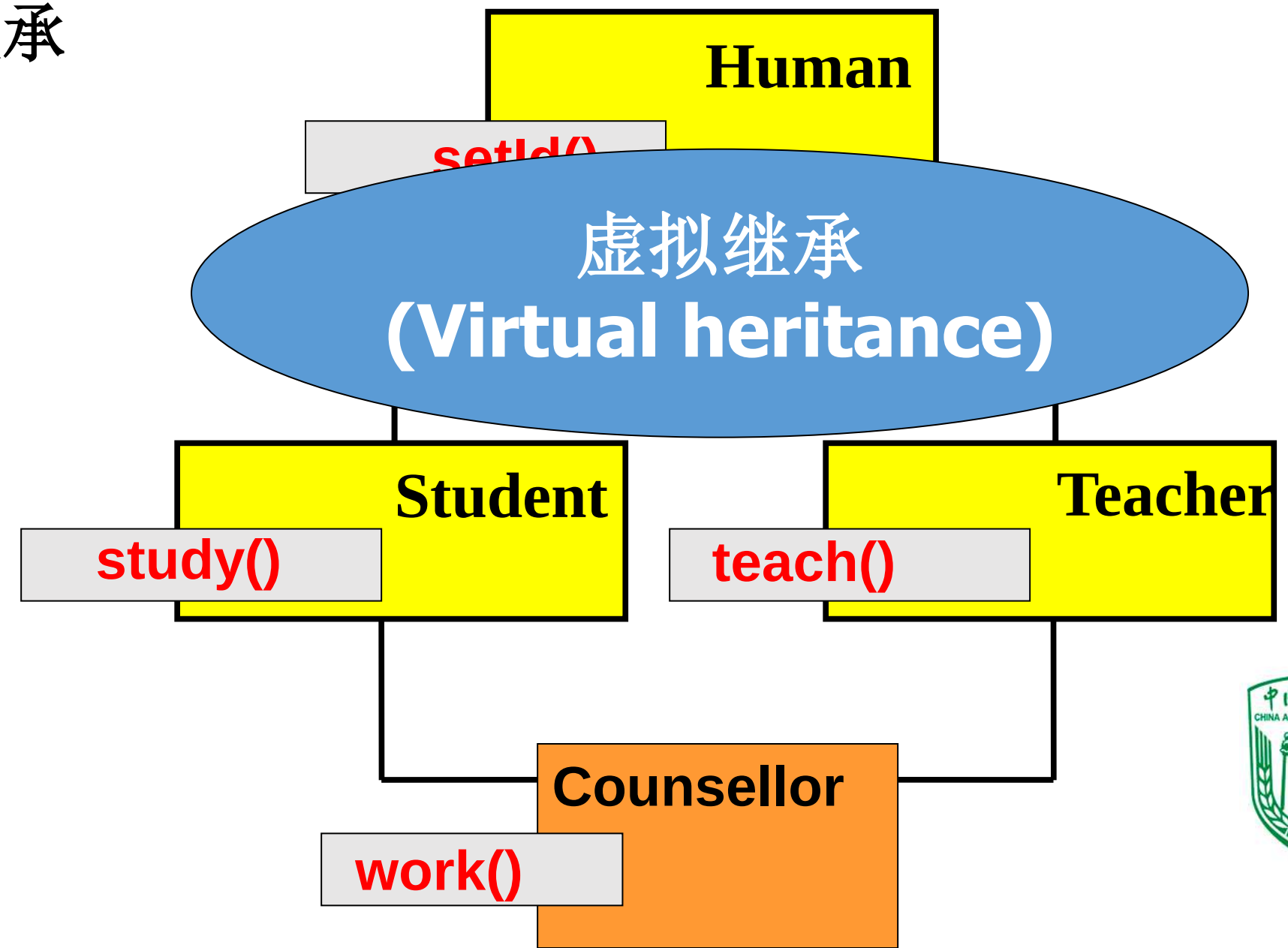

Human
setId()
id
Human
setId()
id
Student
study()
Teacher
teach()
Counsellor
work()

六、多继承

```
#include<iostream.h>
class Human
{protected:     int id;
 public:        Human() {id=0;}
                void setId(int i)
                {id=i;}          };
class Student: public Human
{public:        void study()
                { cout <<"Studying ...\n"; }    };
class Teacher:public Human
{public:        void teach()
                { cout <<"Teaching...\n"; }     };
class Counsellor:public Student, public Teacher
{public:        Counsellor(){}
                void work()
                { cout <<"working...\n"; }
};
void main()
{       Counsellor ss;
        ss.setId(2);
}
```

六、多继承

说明4：　虚拟继承



六、多继承

```
#include<iostream.h>
class Human
{protected:     int id;
 public:        Human() {id=0;}
                void setId(int i)
                {id=i;}          };
class Student: virtual public Human
{public:        void study()
                { cout <<"Studying ...\n"; }     };
class Teacher: virtual public Human
{public:        void teach()
                { cout <<"Teaching...\n"; }      };
class Counsellor:public Student, public Teacher
{public:        Counsellor(){}
                void work()
                { cout <<"working...\n"; }
};
void main()
{       Counsellor ss;
        ss.setId(2);
}
```

六、多继承

```
#include<iostream.h>
class OBJ1
{public:  OBJ1(){ cout <<"OBJ1\n"; }
};
class OBJ2
{public:  OBJ2(){ cout <<"OBJ2\n"; }
};
class Base1
{public:  Base1(){ cout <<"Base1\n"; }
};
class Base2
{public:  Base2(){ cout <<"Base2\n"; }
};
class Base3
{public:  Base3(){ cout <<"Base3\n"; }
};
class Base4
{public:  Base4(){ cout <<"Base4\n"; }
};
```

六、多继承

```
class Derived :public Base1,  virtual public Base2, public Base3, virtual public Base4
{ public:
     Derived() :Base4(), Base3(), Base2(), Base1(), obj2(), obj1()
       {    cout <<"Derived ok.\n";  }
  protected:
     OBJ1 obj1;
     OBJ2 obj2;
};

void main()
{  Derived aa;
   cout <<"This is ok.\n";  }
```

说明5：　多继承的构造顺序

一个具有多继承的派生类对象的构造顺序：

- 任何基类（虚拟或非虚拟）的构造函数按照被继承的顺序构造
- 任何成员对象的构造函数按照声明的顺序构造
- 派生类构造函数

七、基类与派生类的转换

通过公有继承，派生类具备了基类所有的功能，
即凡是基类能够解决的问题，派生类均能解决

在需要基类对象的地方，均可以用公有派生类对象来代替

具体表现在：

- ★ 派生类的对象可以被赋值给基类对象
- ★ 派生类的对象可以初始化基类的引用
- ★ 指向基类对象的指针也可以指向派生类对象

七、基类与派生类的转换

```
#include <iostream>
#include <string>
using namespace std;
class Student
{ public:Student(int n, string nam,float s)
   {        num=n;              name=nam;                    score=s;    }
  void display()
   {        cout<<"num:"<<num<<"\nname:"<<name<<endl;    }
 protected:
   int num;  float score;        string name;};
class Graduate:public Student
{public:   Graduate(int n,string nam,float s,float w):Student(n,nam,s),wage(w){}
           void display()
           {          Student::display();
                      cout<<"wage: "<<wage<<endl;    }
 private:    int wage;            };
void main()
{          Student  studl(1001,"Li",87.5);
           Graduate  gradl(2001,"Wang",98.5,1000);
           Student *pt=&studl;
           pt->display();
           pt=&gradl;
           pt->display(); }
```

小结：

一、继承与派生

二、派生类的声明方式**

三、派生类的构成

四、派生类成员的访问属性***

五、派生类的构造函数和析构函数***

六、多继承*

七、基类与派生类的转换

```
    C(){cout<<"constructing C "<<endl;}
    ~C(){cout<<"destructing C "<<endl;}
};
void main()
{ C c1;
  return 0;
}
```

9. 分别声明 Teacher（教师）类和 Cadre（干部）类，采用多重继承方式由这两个类派生出新类 Teacher_Cadre（教师兼干部）类。要求：

（1）在两个基类中都包含姓名、年龄、性别、地址、电话等数据成员。

（2）在 Teacher 类中还包含数据成员 title（职称），在 Cadre 类中还包含数据成员 post（职务）。在 Teacher_Cadre 类中还包含数据成员 wages（工资）。

（3）对两个基类中的姓名、年龄、性别、地址、电话等数据成员用相同的名字，在引用这些数据成员时，指定作用域。

（4）在类体中声明成员函数，在类外定义成员函数。

（5）在派生类 Teacher_Cadre 的成员函数 show 中调用 Teacher 类中的 display 函数，输出姓名、年龄、性别、职称、地址、电话，然后再用 cout 语句输出职务与工资。

10. 将第 11.8 节中的程序段加以补充完善，使之成为一个完整的程序。在程序中使用继承和组合。在定义 Professor 类对象 prof1 时给出所有数据的初值，然后修改 prof1 的生日数据，最后输出 prof1 的全部最新数据。

派生类的基类，也可以是另一个已定义的类。在一个类中以另一个类的对象作为数据成员的，称为**类的组合**（composition）。

例如，声明 Professor（教授）类是 Teacher（教师）类的派生类，另有一个类 BirthDate（生日），包含 year，month，day 等数据成员。我们可以将教授生日的信息加入到 Professor 类的声明中。如

```
class Teacher                          //声明教师类
{ public:
    ⋮
  private:
    int num;
    string name;
    char sex;
};

class BirthDate                        //声明生日类
{ public:
    ⋮
  private:
    int year;
    int month;
    int day;
};
class Professor:public Teacher         //声明教授类
{ public:
    ⋮
  private:
    BirthDate birthday;                //BirthDate 类的对象作为数据成员
};
```

类的组合和继承一样，是软件重用的重要方式。组合和继承都是有效地利用已有类的资源。但二者的概念和用法不同。**通过继承建立了派生类与基类的关系，它是一种"是"的关系，如"白猫是猫"，"黑人是人"，派生类是基类的具体化实现，是基类中的一种。通过组合建立了成员类与组合类（或称复合类）的关系，在本例中 BirthDate 是成员类，Professor 是组合类（在一个类中又包含另一个类的对象成员）。它们之间不是"是"的关系，而是"有"的关系。**不能说教授（Professor）是一个生日（BirthDate），只能说教授（Professor）有一个生日（BirthDate）的属性。

Professor 类通过继承，从 Teacher 类得到了 num，name，age，sex 等数据成员，通过组合，从 BirthDate 类得到了 year，month，day 等数据成员。**继承是纵向的，组合是横向的。**

如果定义了 Professor 对象 prof1，显然 prof1 包含了生日的信息。通过这种方法有效地组织和利用现有的类，改变了程序人员"一切都要自己干"的工作方式，大大减少了工作量。

如果有以下两个函数：

```
#include <iostream>
#include <string>
using namespace std;
class Teacher
{public:    Teacher(string nam,int a,string t)
            {           name=nam;                   age=a;              title=t;    }
            void display()
            {           cout<<"name:"<<name<<"\nage"<<age<<"\ntitle:"<<title<<endl;          }
 protected: string name;     int age;    string title;          };
class Student
{public:   Student(string nam,char s,float sco)
            {           name1=nam;                  sex=s;                      score=sco;}
            void display1()
            {           cout<<"name:"<<name1<<"\nsex:"<<sex<<"\nscore:"<<score<<endl;          }
            protected:      string name1;                   char sex;                   float score;  };
class Graduate:public Teacher,public Student
{public:   Graduate(string nam,int a,char s,string t,float sco,float w):Teacher(nam,a,t),Student(nam,s,sco),wage(w) {}
            void show( )
            {           cout<<"name:"<<name<<"\nage:"<<age<<"\nsex:"<<sex<<endl;
                        cout<<"score:"<<score<<"\ntitle:"<<title<<"\nwages:"<<wage<<endl;          }
 private:   float wage;  };
void main( )
{           Graduate grad1("Wang-li",24,'f',"assistant",89.5,1234.5);
            grad1.show( );}
```

六、多继承

```
#include <iostream>
#include <string>
using namespace std;
class Person
{public: Person(string nam,char s,int a)
          { name=nam;          sex=s;      age=a;                  }
 protected: string name;       char sex;              int age;    };
class Teacher:virtual public Person
{public:   Teacher(string nam,char s,int a,string t):Person(nam,s,a)
            {title=t;}
 protected:       string title;     };
class Student:virtual public Person
{public:    Student(string nam,char s,int a,float sco):Person(nam,s,a),score(sco){}
  protected:          float score;  };
class Graduate:public Teacher,public Student
{public:    Graduate(string nam,char s,int a,string t,float sco,float w):
            Person(nam,s,a),Teacher(nam,s,a,t),Student(nam,s,a,sco),wage(w){}
            void show( )
             {   cout<<"name:"<<name<<"\nage:"<<age<<"\nsex:"<<sex<<endl;
                 cout<<"score:"<<score<<"\ntitle:"<<title<<"\nwages:"<<wage<<endl;        }
  private:              float wage;  };
 void main( )
{  Graduate grad1("Wang-li",'f',24,"assistant",89.5,1234.5);
  grad1.show( );}
```

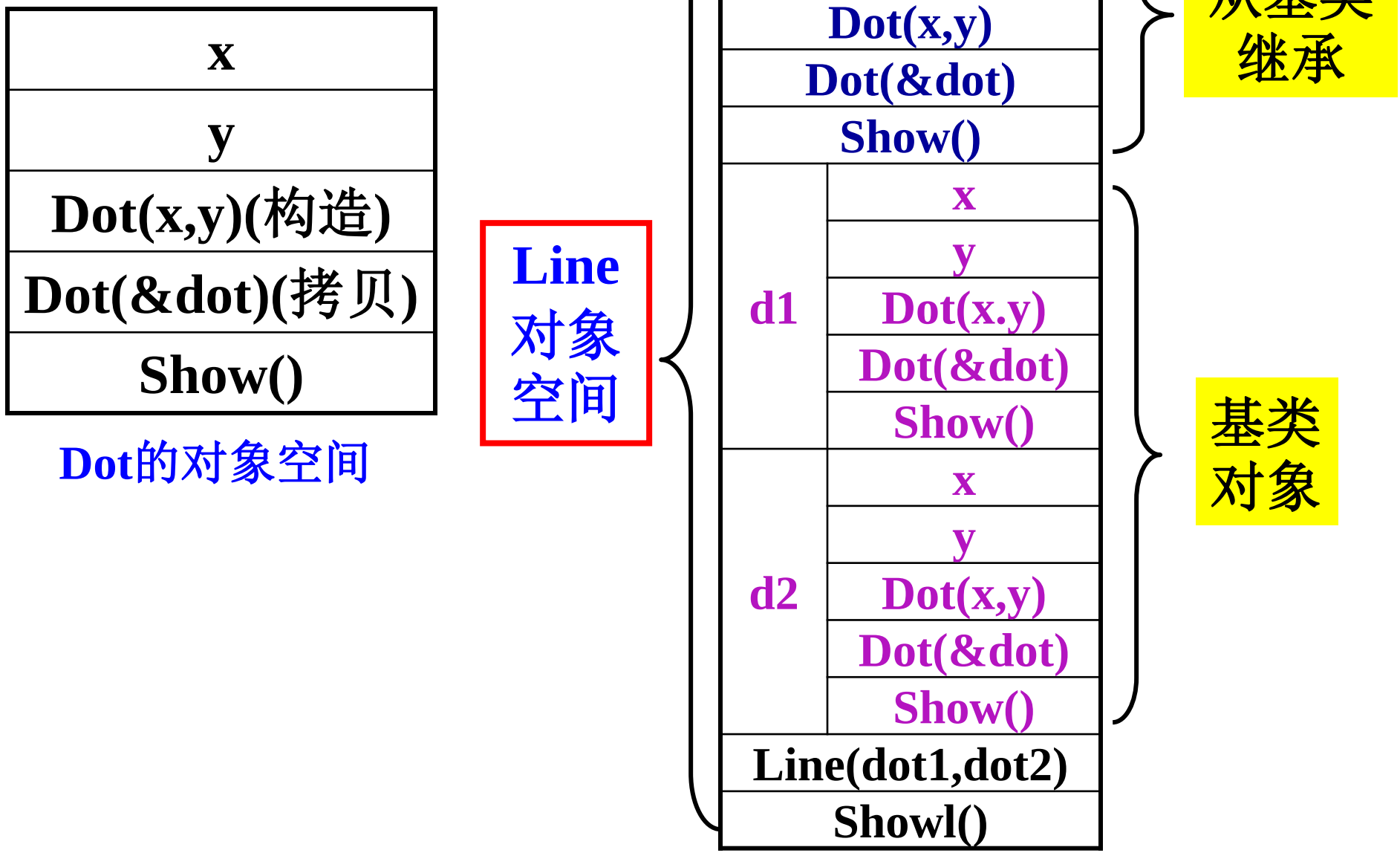
七、基类与派生类的转换

在平面上作两个点，连一直线，求直线的长度和直线中点的坐标。

基类为Dot，有两个公有数据成员，即平面上的坐标（x,y)，同时有构造函数及输出函数。

派生类为Line，有两个基类Dot对象，分别存放两点的坐标，同时，从基类继承了一个Dot数据，存放直线中点的坐标。

七、基类与派生类的转换


x
y
Dot(x,y)(构造)
Dot(&dot)(拷贝)
Show()
Dot的对象空间
Line
对象
空间
x(中点)
y(中点)
Dot(x,y)
Dot(&dot)
Show()
从基类
继承
d1
x
y
Dot(x.y)
Dot(&dot)
Show()
d2
x
y
Dot(x,y)
Dot(&dot)
Show()
基类
对象
Line(dot1,dot2)
Showl()

七、基类与派生类的转换

```
#include <iostream>
#include <string>
using namespace std;
#include "math.h"
class Dot
{public:float x,y;
        Dot(float a=0,float b=0)
        { x=a; y=b;}
        void Show()
        {cout<<"x="<<x<<'\t'<<"y="<<y<<endl;}
};
class Line:public Dot
{       Dot d1,d2;
public:Line(Dot dot1,Dot dot2):d1(dot1),d2(dot2)
        { x=(d1.x+d2.x)/2;    y=(d1.x+d2.y)/2;}
```

```
        void Showl()
        {       cout<<"Dot1:  ";                d1.Show();
                cout<<"Dot2:  ";                d2.Show();
                cout<<"Length="<<sqrt((d1.x-d2.x)*(d1.x-d2.x)+(d1.y-d2.y)*(d1.y-d2.y))<<endl;
                cout<<"Center:  "<<"x="<<x<<'\t'<<"y="<<y<<endl;        }
};
void main()
{   float a,b;
    cout<<"Input Dot1: \n";
    cin>>a>>b;
   Dot dot1(a,b);
   cout<<"Input Dot2: \n";
   cin>>a>>b;
   Dot dot2(a,b);
   Line line(dot1,dot2);
   line.Showl();                }
```

一、继承与派生

例题 5-1

```
#include<iostream>
#include "string"
using namespace std;
class Student
{public:   void get_value()
           {cin>>num>>name>>sex;}
           void display()
           {          cout<<" num: "<<num<<endl;
                      cout<<" name: "<<name<<endl;
                      cout<<" sex:"<<sex<<endl;     }
 private: int num;
           string name;
           char sex;
};
class Student1:public Student
{public:   void get_value_1()
           {          cin>>age>>addr;                }
           void display_1()
           {          cout<<"age:"<<age<<endl;
           cout<<"address:"<<addr<<endl; }
private:   int age;
           string addr;};
```

```
void main()
{          Student1 stud;
           stud.get_value();
           stud.get_value_1();
           stud.display();
           stud.display_1();}
```

谢　谢